

Instructions for Creating Lightweight-YOLO

1 Experimental environment

1.1 Experimental parameters

Table 1 Experimental parameters

Environment configuration	model	Remark
CPU	AMD EPYC 7642 48-Core Processor	
GPU	NVIDIA GeForce RTX 3090	Large 24 GB of video memory
Computer operating system	Ubuntu 20.04.5 LTS	
Deep learning framework	torch-2.0.0+cu117	

1.2 Experimental environment deployment

conda environment

```
conda create -n yolov8 python=3.8

conda activate yolov8

# Determine the overall code path
pip install -r requirements.txt
```

2 YOLOv8 algorithm baseline model

The structure diagram of YOLOv8n is shown in Figure 1.

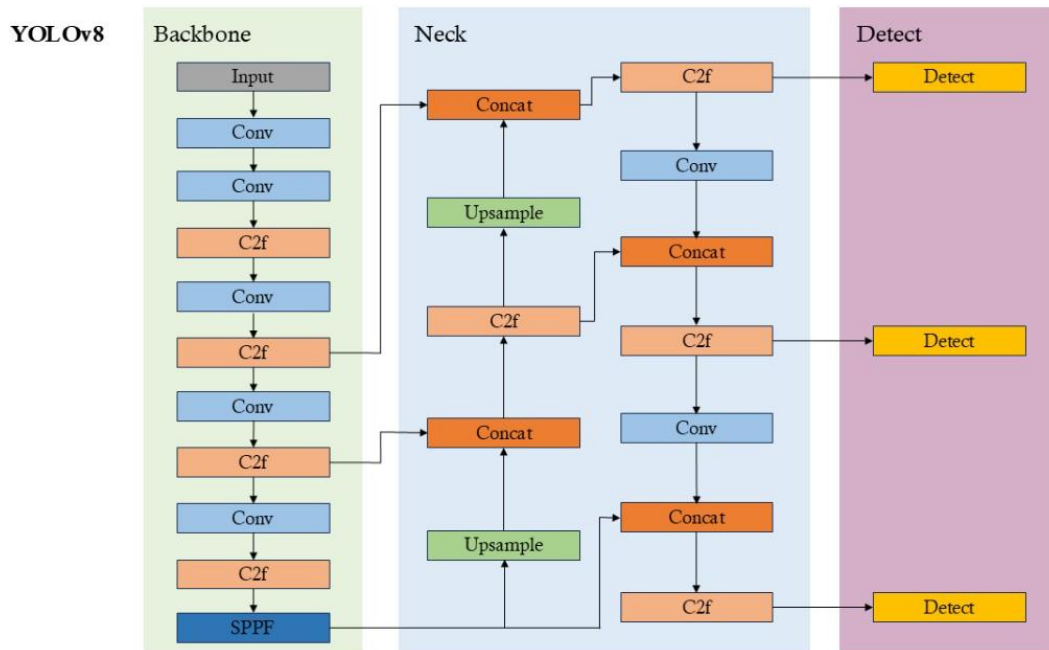


Figure 1 YOLOv8n structure diagram

2.1 baseline core code

```
# Ultralytics YOLO 🚀, AGPL-3.0 license
# YOLO11 object detection model with P3-P5 outputs. For Usage examples see
https://docs.ultralytics.com/tasks/detect
```

```
# YOLOv8.0n backbone
backbone:
  # [from, repeats, module, args]
  - [-1, 1, Conv, [64, 3, 2]] # 0-P1/2
  - [-1, 1, Conv, [128, 3, 2]] # 1-P2/4
  - [-1, 3, C2f, [128, True]]
  - [-1, 1, Conv, [256, 3, 2]] # 3-P3/8
  - [-1, 6, C2f, [256, True]]
  - [-1, 1, Conv, [512, 3, 2]] # 5-P4/16
  - [-1, 6, C2f, [512, True]]
  - [-1, 1, Conv, [1024, 3, 2]] # 7-P5/32
  - [-1, 3, C2f, [1024, True]]
  - [-1, 1, SPPF, [1024, 5]] # 9

# YOLOv8.0n head
head:
  - [-1, 1, nn.Upsample, [None, 2, 'nearest']]
  - [[-1, 6], 1, Concat, [1]] # cat backbone P4
  - [-1, 3, C2f, [512]] # 12

  - [-1, 1, nn.Upsample, [None, 2, 'nearest']]
  - [[-1, 4], 1, Concat, [1]] # cat backbone P3
  - [-1, 3, C2f, [256]] # 15 (P3/8-small)

  - [-1, 1, Conv, [256, 3, 2]]
  - [[-1, 12], 1, Concat, [1]] # cat head P4
  - [-1, 3, C2f, [512]] # 18 (P4/16-medium)

  - [-1, 1, Conv, [512, 3, 2]]
  - [[-1, 9], 1, Concat, [1]] # cat head P5
  - [-1, 3, C2f, [1024]] # 21 (P5/32-large)

  - [[15, 18, 21], 1, Detect, [nc]] # Detect(P3, P4, P5)
```

2.2 baseline test

Figure 2 shows the test results of YOLOv8n on the Locount dataset.

```
(yolov8) root@autodl-container-d83b46aa62-0763ef7f:~/autodl-tmp/YOLOv8_locount/yolov8# python val.py
Ultralytics YOLOv8.0.135 Python-3.8.17 torch-2.0.0+cu117 CUDA:0 (NVIDIA GeForce RTX 3090, 24260MiB)
YOLOv8n summary (fused): 168 layers, 3343451 parameters, 0 gradients, 13.1 GFLOPs
val: Scanning /root/autodl-tmp/YOLOv8_locount/datasets/test/labels.cache... 500 images, 0 backgrounds, 0 corrupt: 100%|██████████| 500/500 [00:00<?, ?it/s]

```

Class	Images	Instances	Box(P)	R	mAP50	mAP50-95)
all	500	7991	0.708	0.502	0.600	0.395
Jacket	500	190	0.653	0.463	0.561	0.324
Trousers	500	172	0.758	0.651	0.76	0.515
Toothpaste	500	153	0.467	0.405	0.419	0.214
Children shoes	500	16	1	0	0	0

```
Speed: 0.3ms preprocess, 4.0ms inference, 0.0ms loss, 4.8ms postprocess per image
Results saved to /root/autodl-tmp/YOLOv8_locount/yolov8/runs/train2
```

Figure 2 YOLOv8n test results on the Locount dataset

3 Improvement point 1 DWConv

3.1 Improvement point 1: Model principle

In the task of detecting objects in a store or supermarket, the targets include 109 categories of goods. These goods are diverse in type and appearance. Traditional convolutional operations may consume excessive computational resources and may affect efficiency for tasks with high real-time requirements. To optimize this issue, YOLOv8's DWConv (depthwise convolution) replaces the standard convolutional operation, thereby achieving model lightweighting, which reduces the number of model parameters and computational cost while maintaining the same accuracy.

一、DWConv's design goals:

DWConv's design is based on the following core objectives:

1. Reduced computation and parameter count: Compared to standard convolution, depthwise convolution reduces the computational cost per convolution kernel. It performs convolution independently for each input channel, significantly reducing the computational complexity and parameter count of the convolutional layer.
2. Improved efficiency and real-time performance: Since DWConv performs convolution operations only on each input channel, it reduces a significant amount of computation, making it suitable for real-time detection tasks with limited computing resources. This improves detection efficiency and allows for more complex and efficient deployment scenarios.
3. Accuracy Preservation: Although DWConv significantly reduces computational cost, it is still able to retain and extract sufficient features, thus not affecting the model's accuracy. This is especially important in tasks such as store and supermarket item detection, where accuracy is crucial for product recognition.

二、DWConv module principle:

In YOLOv8, DWConv is a depthwise convolution operation that applies a convolution kernel independently to each channel of the input, rather than applying a shared kernel to all input channels. Its basic principle can be summarized in the following steps:

1. Depthwise convolution operation: When DWConv convolves the input feature map, each convolutional kernel processes only a single input channel, instead of processing all channels simultaneously as in traditional convolution. This means that for each channel, the convolutional kernel only performs local perception within that channel, reducing computational complexity.
2. Reduced computational and parameter count: In standard convolution, the kernel size is (C_{in}, C_{out}, H, W) , where C_{in} is number of input channels, C_{out} is number of output channels. The number of convolution kernels is C_{out} . In DWConv, the number of convolutional kernels is equal to the number of input channels is C_{in} . The convolution is computed only once per channel, significantly reducing computational cost. For each input channel, the kernel size is $(1, 1)$, This further reduces the amount of computation.
3. Batch Normalization and Activation: After DWConv, a **Batch Normalization** layer is added to standardize the convolution results, remove potential biases, and improve training stability. Subsequently, through the activation function (如 'SiLU') The output feature map is subjected to a nonlinear transformation to enhance the nonlinear expressive power of the model. Among them, activation function 'SiLU' (Sigmoid Linear Unit) It has smooth nonlinear characteristics, which can improve the model's ability to capture complex features.*

三、Application of DWConv in YOLOv8

In YOLOv8, DWConv is integrated into the **Neck** network structure, replacing traditional convolutional operations. This optimization allows YOLOv8 to maintain good detection performance while reducing computational complexity, making it particularly suitable for object detection tasks in stores and supermarkets. In environments with numerous object categories and complex backgrounds, the DWConv module can efficiently extract key features of goods, ensuring the accuracy of object detection.

The core code of the DWConv module is shown above. It inherits from the standard convolutional layer class

and uses `math.gcd` to determine the 'groups' of each convolutional kernel, thereby implementing depthwise convolution operations. This allows the module to execute more efficiently and reduce computational burden, ultimately achieving a reduction in the model's computational and memory overhead while maintaining detection accuracy.

By replacing traditional convolutions with **DWConv**, YOLOv8 achieves a **lightweight design** in the **store/supermarket item detection** task, significantly reducing computation and parameter count without significantly impacting detection accuracy. The DWConv module not only improves model efficiency but also achieves a good balance between efficiency and accuracy, providing effective technical support for real-time object detection tasks.

The structural diagram of DWConv is shown in Figure 3.

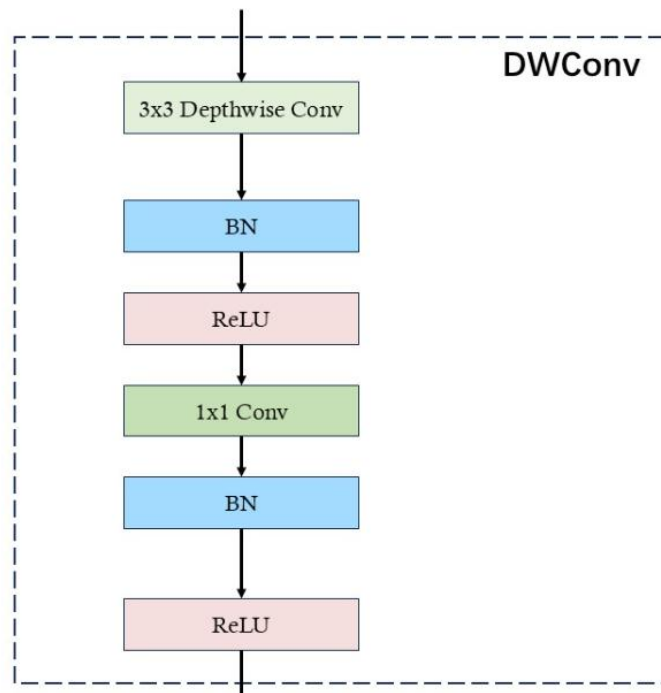


Figure 3 DWConv structural diagram

The model structure of YOLOv8 after adding DWConv is shown below:

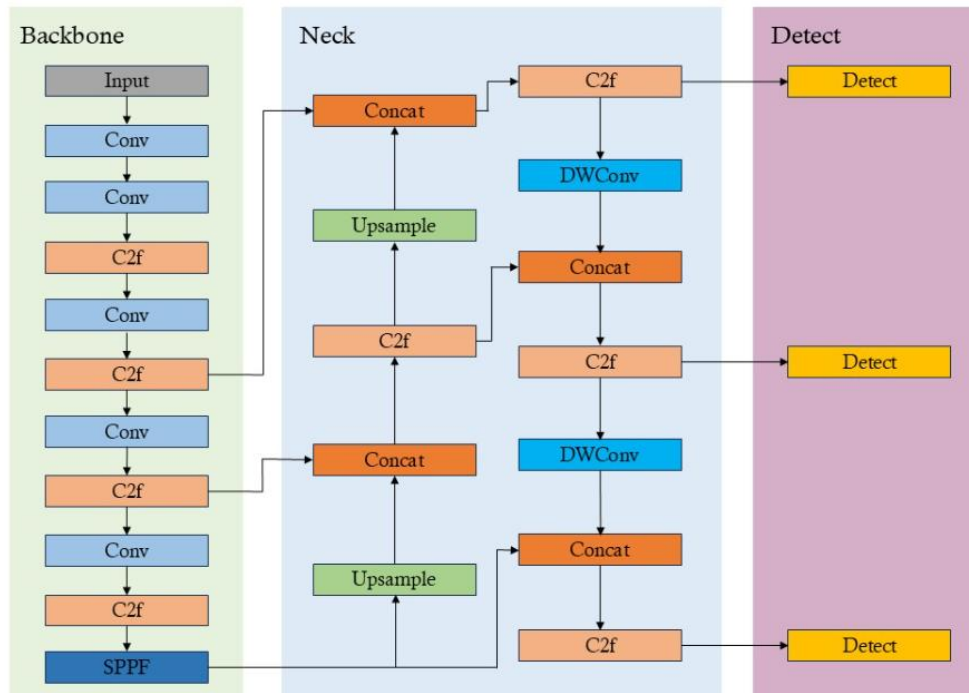


Figure 4 YOLOv8n+DWConv structure diagram

3.2 Improvement Point 1 Core Code

```
import torch
import torch.nn as nn

class Conv(nn.Module):
    """Standard convolution with args(ch_in, ch_out, kernel, stride, padding, groups, dilation,
    activation)."""
    default_act = nn.SiLU() # default activation

    def __init__(self, c1, c2, k=1, s=1, p=None, g=1, d=1, act=True):
        """Initialize Conv layer with given arguments including activation."""
        super().__init__()
        self.conv = nn.Conv2d(c1, c2, k, s, autopad(k, p, d), groups=g, dilation=d, bias=False)
        self.bn = nn.BatchNorm2d(c2)
        self.act = self.default_act if act is True else act if isinstance(act, nn.Module) else nn.Identity()

    def forward(self, x):
        """Apply convolution, batch normalization and activation to input tensor."""
        return self.act(self.bn(self.conv(x)))

    def forward_fuse(self, x):
        """Perform transposed convolution of 2D data."""
        return self.act(self.conv(x))

class DWConv(Conv):
    """Depth-wise convolution."""

    def __init__(self, c1, c2, k=1, s=1, d=1, act=True): # ch_in, ch_out, kernel, stride, dilation,
        activation
        super().__init__(c1, c2, k, s, g=math.gcd(c1, c2), d=d, act=act)
```

3.3 Added Improvement Point 1: Testing

The test results of YOLOv8n+DWConv on the Locount dataset are shown in Figure 5.

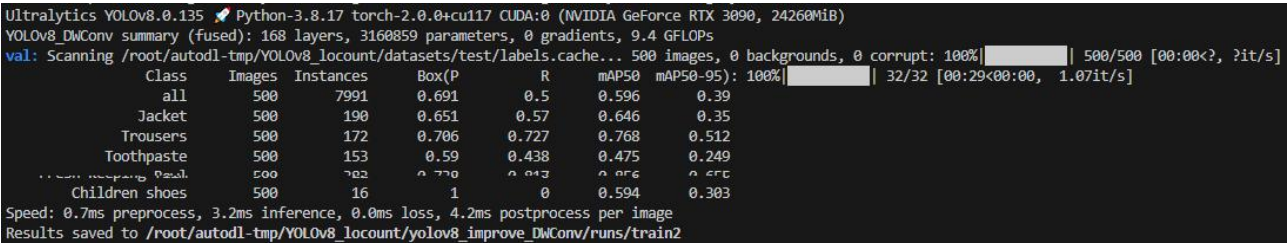


Figure 5 YOLOv8n+DWConv test results on the Locount dataset

4 Improvement Points 2 CSPHet-CBAM

4.1 Improvement Point 2: Model Principle

In the task of object detection in stores and supermarkets, YOLOv8 is used to identify up to 109 categories of goods. To improve the model's lightweight nature, reduce computational overhead, and maintain high accuracy, we optimized the Neck network in YOLOv8 by replacing the original C2f module with CSPHet-CBAM. This replacement not only reduced the number of model parameters and computational cost but also maintained the same detection accuracy.

一、Design Goals of CSPHet-CBAM:

CSPHet-CBAM is designed to optimize the following aspects:

1. Improving computational efficiency:

By replacing the C2f module, CSPHet-CBAM introduces the **HetConv** (heterogeneous convolution) structure, which combines multiple convolution operations of different sizes, thereby reducing computational cost, especially when the number of channels is large.

2. Enhancing Feature Representation Capability:

By combining the channel and spatial attention mechanisms of **CBAM** (Convolutional Block Attention Module), CSPHet-CBAM enhances the model's ability to focus on important features, especially in the detection of complex backgrounds and small objects, and can more effectively extract key object features.

3. Flexible structural design:

Through the CSP (Cross-Stage Partial) design, CSPet-CBAM incorporates a partial residual structure, enabling features to flow more effectively, reducing the gradient vanishing problem, and enhancing the robustness of the model.

二、CSPHet-CBAM Module Principles:

CSPHet-CBAM is a structure combining **CSPHet-Bottleneck** and **CBAM**, and its specific working principle is as follows:

CBAM (Channel and spatial attention mechanisms) :

The CBAM module enhances the model's attention to important regions, especially key objects and local features, through two attention mechanisms—channel attention and spatial attention.

Channel attention adaptively adjusts the weights of different channels, strengthening focus on important channels. In item detection in stores and supermarkets, certain items may have strong features in certain channels; channel attention helps the model focus on this information.

Spatial attention, on the other hand, generates a spatial attention map based on the correlation of spatial locations, thereby focusing on key areas of an object, especially when the object is partially occluded or in a complex background.

By combining CBAM, CSPHet-CBAM can dynamically select and weight input features, improving the model's sensitivity to product details.

HetConv:

HetConv is an innovative convolutional structure that combines 1x1 and 3x3 convolutional operations to adapt to different feature scales. Specifically:

1x1 convolutional: This is typically used to reduce the number of channels and reduce computational burden by lowering the dimensionality.

3x3 convolutional: Used to extract local features while preserving spatial information.

By combining these two convolutional operations, the HetConv structure can reduce computational cost while preserving rich feature representation capabilities, especially when dealing with high-dimensional features, it can significantly reduce computational overhead.

CSPHet-Bottleneck structure:

CSPHet-Bottleneck is the core structure in CSPHet-CBAM. It adopts the idea of **CSP**, which divides the input features into multiple parts, processes them through different paths, and then merges them into the output.

Channel Segmentation: This involves dividing the input features into different parts and feeding them into different convolutional paths for processing.

Two convolution operations: HetConv is used to perform convolution operations on each path, extracting features of different dimensions.

Feature Fusion: Finally, the features extracted from different paths are concatenated and fused using a 1x1 convolution to output the final feature map.

The CSPHet-Bottleneck structure enables efficient computation during feature extraction while avoiding excessive information loss.

三、Application of CSPHet-CBAM in YOLOv8:

In the YOLOv8 **Neck** network, the original C2f module is replaced with **CSPHet-CBAM**, achieving a lightweight optimization for object detection tasks. By combining **CSPHet-Bottleneck** and **CBAM**, this improved structure can efficiently extract key features of items in stores and supermarkets, handle diverse product categories, and maintain high efficiency in real-time detection.

Workflow:

1. **Input:** The input feature map is extracted to the intermediate layer through the convolutional layer.
2. **Feature Segmentation and Processing:** The feature map is segmented into different parts and then fed into HetConv for processing.
3. **CBAM Weighting:** The CBAM module weights important features to enhance the target detection capability.
4. **Feature Fusion and Output:** The fused feature map is finally output through a 1x1 convolution.

This improvement significantly enhances YOLOv8's performance in store and supermarket item inspection while meeting the requirements of lightweight design and high precision.

By replacing the C2f module with **CSPHet-CBAM**, YOLOv8 achieves a **lightweight design** in the store/supermarket object detection task, while reducing computational and memory overhead. CBAM's attention mechanism and HetConv's heterogeneous convolutional structure enhance the model's feature extraction capabilities, maintaining detection accuracy in complex backgrounds and multi-object environments.

Figure 6 shows the structural diagrams of HetConv and CBAM. Figure 7 shows the structural diagrams of CSPHet-CBAM.

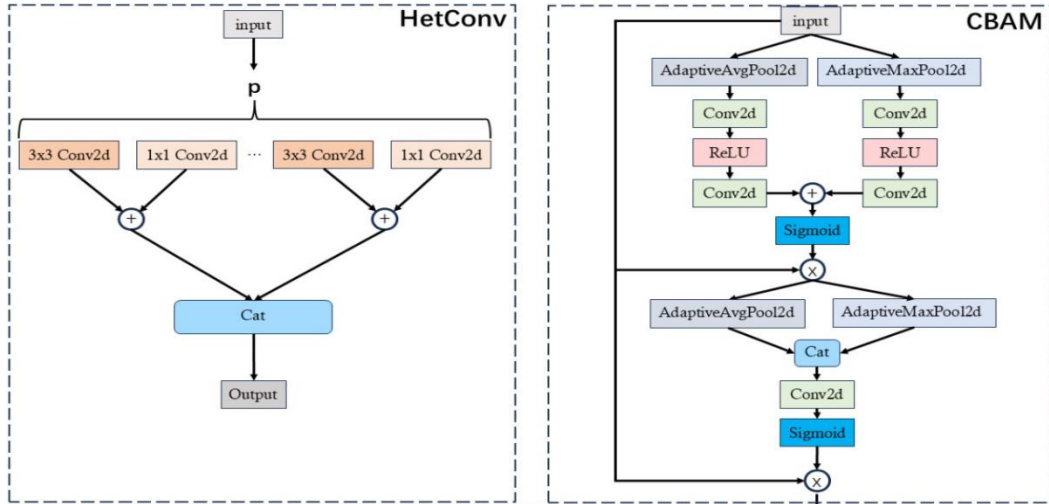


Figure 6 Structural diagrams of HetConv and CBAM

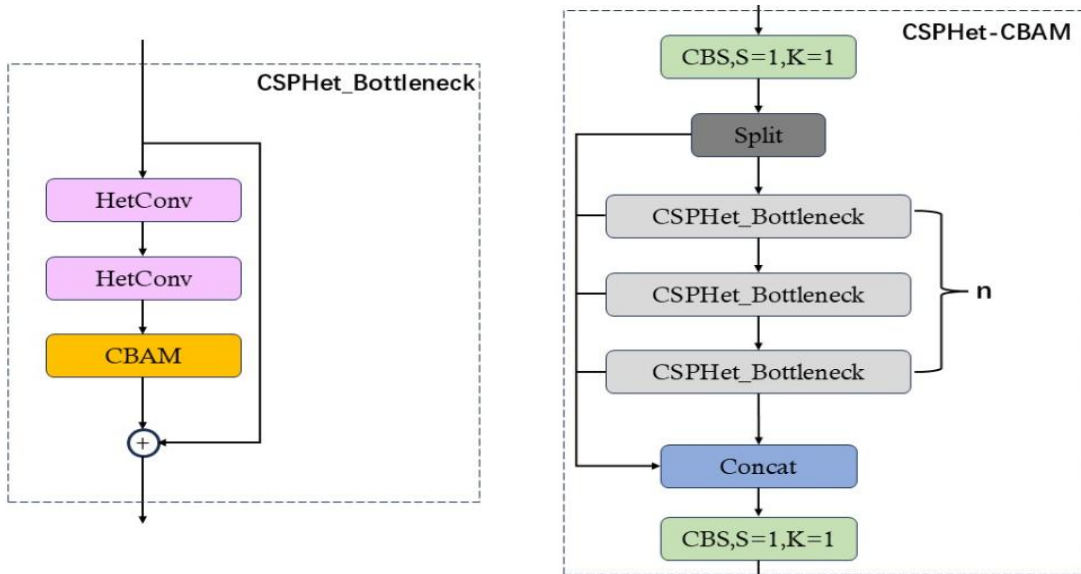


Figure 7 Structural diagrams of CSPHet-CBAM

The model structure after adding CSPHet-CBAM to YOLOv8+DWConv is shown below:

YOLOv8+DWConv+
CSPHet-CBAM

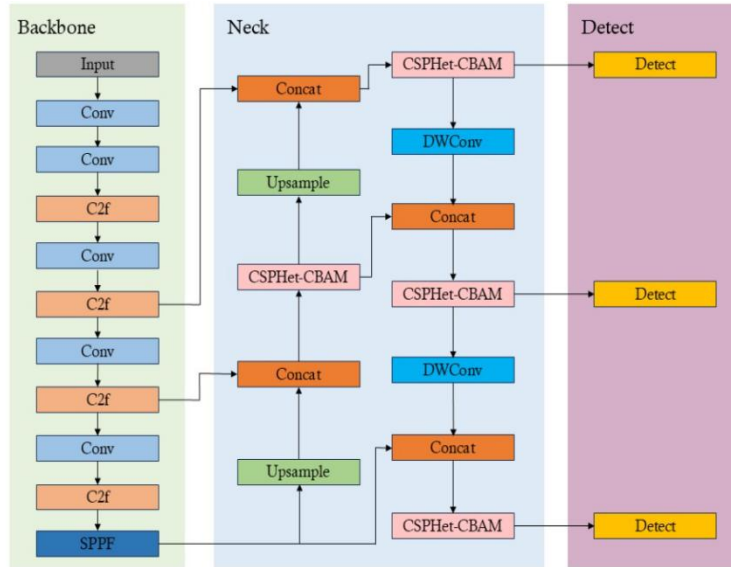


Figure 8 Structural diagram of YOLOv8n+DWConv+CSPHet-CBAM

4.2 Improvement Point 2: Core Code

```
import torch
import torch.nn as nn

class ChannelAttention(nn.Module): #Channel attention mechanism
    def __init__(self, in_planes, scaling=16):#scaling refers to the scaling ratio.,
        # This is used to control the number of neurons in the neural
        # network between two fully connected layers. It is generally set to 16, but can be fine-tuned as needed.
        super(ChannelAttention, self).__init__()
        self.avg_pool = nn.AdaptiveAvgPool2d(1)
        self.max_pool = nn.AdaptiveMaxPool2d(1)

        self.fc1 = nn.Conv2d(in_planes, in_planes // scaling, 1, bias=False)
        self.relu1 = nn.ReLU()
        self.fc2 = nn.Conv2d(in_planes // scaling, in_planes, 1, bias=False)

        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        avg_out = self.fc2(self.relu1(self.fc1(self.avg_pool(x))))
        max_out = self.fc2(self.relu1(self.fc1(self.max_pool(x))))
        out = avg_out + max_out
        out = self.sigmoid(out)
        return out
```

```

class SpatialAttention(nn.Module): #Spatial attention mechanism
    def __init__(self, kernel_size=7):
        super(SpatialAttention, self).__init__()

        assert kernel_size in (3, 7), 'kernel size must be 3 or 7'
        padding = 3 if kernel_size == 7 else 1
        self.conv1 = nn.Conv2d(2, 1, kernel_size, padding=padding, bias=False)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        avg_out = torch.mean(x, dim=1, keepdim=True)
        max_out, _ = torch.max(x, dim=1, keepdim=True)
        x = torch.cat([avg_out, max_out], dim=1)
        x = self.conv1(x)
        x = self.sigmoid(x)
        return x

```

```

class CBAM_Attention(nn.Module):
    def __init__(self, channel, scaling=16, kernel_size=7):
        super(CBAM_Attention, self).__init__()
        self.channelattention = ChannelAttention(channel, scaling=scaling)
        self.spatialattention = SpatialAttention(kernel_size=kernel_size)

    def forward(self, x):
        x = x * self.channelattention(x)
        x = x * self.spatialattention(x)
        return x

```

```

class CBAM_Attention(nn.Module):
    def __init__(self, channel, scaling=16, kernel_size=7):
        super(CBAM_Attention, self).__init__()
        self.channelattention = ChannelAttention(channel, scaling=scaling)
        self.spatialattention = SpatialAttention(kernel_size=kernel_size)

    def forward(self, x):
        x = x * self.channelattention(x)
        x = x * self.spatialattention(x)
        return x

```

```

class HetConv(nn.Module):

    def __init__(self, input_channels, output_channels, stride=1, p=4):
        """
        Initialize the HetConv class.
        :param input_channels: the number of input channels
        :param output_channels: the number of output channels
        :param stride: convolution stride
        :param p: the value of P used in HetConv
        """
        super(HetConv, self).__init__()
        self.p = p
        self.input_channels = input_channels
        self.output_channels = output_channels
        self.filters = nn.ModuleList()
        self.convolution_1x1_index = []

        # Compute the indices of input channels fed to 1x1 convolutional kernels in all filters.
        # These indices of input channels are also the indices of the 1x1 convolutional kernels in the
        filters.

```

```

# This is only executed when the HetConv class is created,
# and the execution time is not included during inference.
for i in range(self.p):
    self.convolution_1x1_index.append(self.compute_convolution_1x1_index(i))
# Build HetConv filters.
for i in range(self.p):
    self.filters.append(self.build_HetConv_filters(stride, p))

def compute_convolution_1x1_index(self, i):
    """
    Compute the indices of input channels fed to 1x1 convolutional kernels in the i-th branch of filters
    (i=0, 1, 2,..., P-1). The i-th branch of filters consists of the {i, i+P, i+2P,..., i+N-P}-th filters.
    :param i: the i-th branch of filters in HetConv
    :return: return the required indices of input channels
    """
    index = [j for j in range(0, self.input_channels)]
    # Remove the indices of input channels fed to 3x3 convolutional kernels in the i-th branch of
    filters.
    while i < self.input_channels:
        index.remove(i)
        i += self.p
    return index

def build_HetConv_filters(self, stride, p):
    """
    Build N/P filters in HetConv.
    :param stride: convolution stride
    :param p: the value of P used in HetConv
    :return: return N/P HetConv filters
    """
    temp_filters = nn.ModuleList()

    # nn.Conv2d arguments: nn.Conv2d(input_channels, output_channels, kernel_size, stride, padding)
    temp_filters.append(nn.Conv2d(self.input_channels//p, self.output_channels//p, 3, stride, 1,
    bias=False))
    temp_filters.append(nn.Conv2d(self.input_channels-self.input_channels//p, self.output_channels//p,
    1, stride, 0, bias=False))
    return temp_filters

def forward(self, input_data):
    """
    Define how HetConv processes the input images or input feature maps.
    :param input_data: input images or input feature maps
    :return: return output feature maps
    """
    output_feature_maps = []
    # Loop P times to get output feature maps. The number of output feature maps = the batch size.
    for i in range(0, self.p):

        # M/P HetConv filter kernels perform the 3x3 convolution and output to N/P output channels.
        output_feature_3x3 = self.filters[i][0](input_data[:, i::self.p, :, :])
        # (M-M/P) HetConv filter kernels perform the 1x1 convolution and output to N/P output channels.
        output_feature_1x1 = self.filters[i][1](input_data[:, self.convolution_1x1_index[i], :, :])

        # Obtain N/P output feature map channels.
        output_feature_map = output_feature_1x1 + output_feature_3x3

        # Append N/P output feature map channels.
        output_feature_maps.append(output_feature_map)

```



```

# Get the batch size, number of output channels (N/P), height and width of output feature map.
N, C, H, W = output_feature_maps[0].size()
# Change the value of C to the number of output feature map channels (N).
C = self.p * C
# Arrange the output feature map channels to make them fit into the shifted manner.
return torch.cat(output_feature_maps, 1).view(N, self.p, C//self.p, H, W).permute(0, 2, 1, 3,
4).contiguous().view(N, C, H, W)

class CSPHet_Bottleneck(nn.Module):
    def __init__(self, dim):
        super().__init__()
        self.cbam = CBAM_Attention(dim)
        self.DualPConv = nn.Sequential(HetConv(dim, dim), HetConv(dim, dim))

    def forward(self, x):
        return self.cbam(self.DualPConv(x))

class CSPHet_CBAM(nn.Module):
    # CSP Bottleneck with 2 convolutions
    def __init__(self, c1, c2, n=1, shortcut=False, g=1, e=0.5): # ch_in, ch_out, number, shortcut, groups,
expansion
        super().__init__()
        self.c = int(c2 * e) # hidden channels
        self.cv1 = Conv(c1, 2 * self.c, 1, 1)
        self.cv2 = Conv((2 + n) * self.c, c2, 1) # optional act=FReLU(c2)
        self.m = nn.ModuleList(CSPHet_Bottleneck(self.c) for _ in range(n))

    def forward(self, x):
        y = list(self.cv1(x).split((self.c, self.c), 1))
        y.extend(m(y[-1]) for m in self.m)
        return self.cv2(torch.cat(y, 1))

```

4.3 Added improvement point 2 to the test.

Figure 9 shows YOLOv8n+DWConv+CSPHet-CBAM on the Locount dataset.

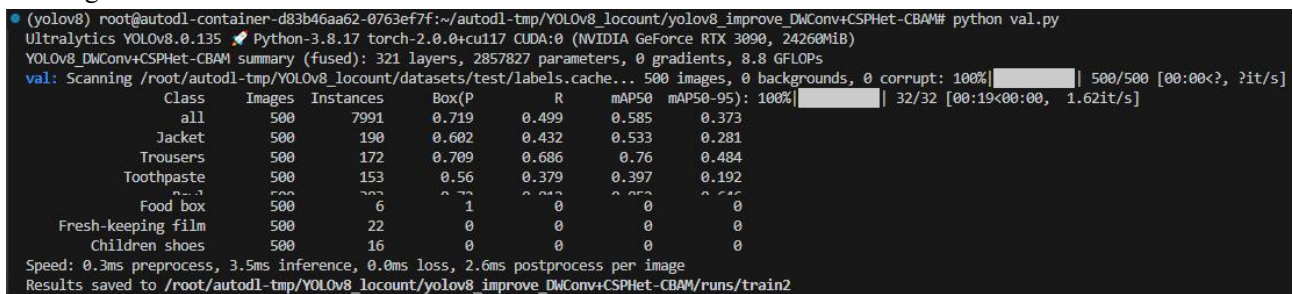


Figure 9 Results of YOLOv8n+DWConv+CSPHet-CBAM on the Locount dataset

5 Improvement point 3 ADown

5.1 Improvement Point 3: Model Principle

In the store/supermarket item detection task, YOLOv8 was used to identify 109 items. To achieve lightweight improvements (reducing the number of parameters and computational cost) while ensuring no decrease in accuracy, and even a slight increase, we replaced the last four traditional convolutional layers in the

Backbone network of YOLOv8 with **ADown** (Adaptive Downsampling module). This replacement not only optimized the computational efficiency of the model but also improved the performance of real-time object detection.

一、ADown's design goals:

1. Reduce computational load and number of parameters:

By replacing traditional convolution operations, ADown uses a specific downsampling method combined with convolution to reduce unnecessary computation and parameters, thereby improving the model's computational efficiency, especially when dealing with high-dimensional input features.

2. Enhance feature representation ability:

Although it reduces computational complexity, ADown still retains the ability to extract key features and effectively capture detailed information, especially in object details and complex backgrounds.

3. Maintaining accuracy and real-time performance:

While maintaining a lightweight design, ADown utilizes an adaptive downsampling method, enabling the model to accurately capture important features while reducing computational load, thereby maintaining the model's detection accuracy and even achieving slight improvements in some cases.

二、ADown Module Principles:

1. ADown's core design idea is to compress input features through **downsampling** operations and then extract effective features using convolution. Its structure comprises two key steps:

2. Input feature segmentation:

ADown splits the input feature map into two parts along the channel dimension, processing each part separately. This operation effectively reduces computation and optimizes feature extraction efficiency.

3. Adaptive downsampling and convolution processing:

The first part uses 3x3 convolution and downsampling operation with a stride of 2. (`'avg_pool2d'`) perform feature reduction.

The second part first uses 3x3 max pooling. (`'max_pool2d'`) spatial dimensionality reduction, Then, the channel information is further compressed through 1x1 convolution.

Finally, ADown concatenates the two processed feature maps to preserve the original information and enhance feature representation capabilities.

ADown Detailed Process:

1. Feature Segmentation: The input feature map `'x'` is divided into two parts: `'x1'` and `'x2'`. This step uses the `'torch.chunk()'` function to split the input into two parts along the channel dimension, reducing the complexity of subsequent calculations.

2. First part of the processing (x1):

Perform a 3x3 convolution on `'x1'` and downsample it with a stride of 2 to reduce spatial resolution.

Using `avg_pool2d` to perform average pooling on the entire input makes the downsampling operation smoother.

3. Part Two Processing (x2):

Perform 3x3 max pooling on `'x2'` with a stride of 2 to further reduce the spatial dimension.

Then, the pooled feature map is compressed in channels using 1x1 convolution.

4. Concatenation Feature: Finally, `'torch.cat()'` concatenates the processed `'x1'` and `'x2'` together to form the final output.

三、Application of ADown in YOLOv8:

In the **Backbone** network of YOLOv8, replacing traditional convolutional layers with **ADown** significantly

improves the model's computational efficiency while maintaining stable accuracy. Through the lightweight design of ADown, YOLOv8 demonstrates higher inference speed and lower computational cost in store and supermarket object detection tasks.

Reduced computational cost: ADown reduces computational cost through efficient downsampling and convolution operations, enabling the model to run more smoothly in real-time applications.

Stable Accuracy: Through precise feature extraction, ADown ensures the accuracy of object detection, especially in scenes with dense items and complex backgrounds, where the model can reliably identify 109 categories of goods.

By replacing the traditional convolutional layers in YOLOv8 with **ADown**, we successfully achieved a **lightweight improvement**, significantly reducing computational cost and parameter count while maintaining accuracy. ADown reduces redundant computations while retaining crucial feature extraction capabilities, providing efficient technical support for store and supermarket item detection tasks.

The ADown Structure is shown in Figure 10.

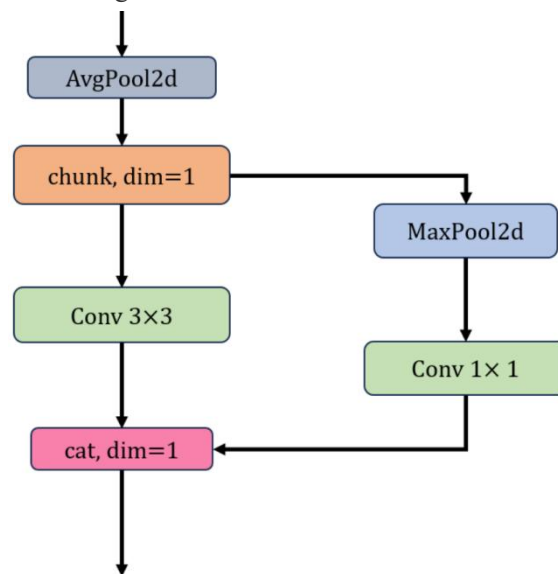


Figure 10 ADown Structure

The model structure after adding ADown to YOLOv8+DWConv+CSPHet-CBAM is shown below:

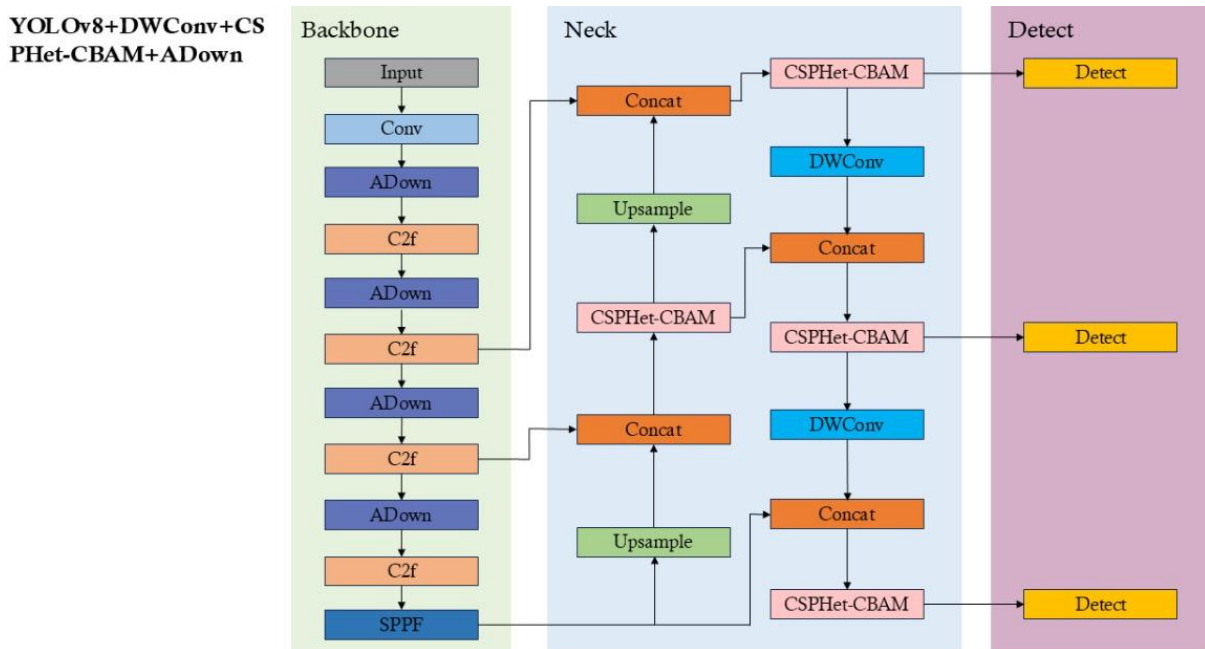


Figure 11 Structural diagram of YOLOv8n+DWConv+CSPHet-CBAM+ADown

5.2 Improvement point 3: Core code

```
import torch
import torch.nn as nn

class Conv(nn.Module):
    """Standard convolution with args(ch_in, ch_out, kernel, stride, padding, groups, dilation,
    activation)."""
    default_act = nn.SiLU() # default activation

    def __init__(self, c1, c2, k=1, s=1, p=None, g=1, d=1, act=True):
        """Initialize Conv layer with given arguments including activation."""
        super().__init__()
        self.conv = nn.Conv2d(c1, c2, k, s, autopad(k, p, d), groups=g, dilation=d, bias=False)
        self.bn = nn.BatchNorm2d(c2)
        self.act = self.default_act if act is True else act if isinstance(act, nn.Module) else nn.Identity()

    def forward(self, x):
        """Apply convolution, batch normalization and activation to input tensor."""
        return self.act(self.bn(self.conv(x)))

    def forward_fuse(self, x):
        """Perform transposed convolution of 2D data."""
        return self.act(self.conv(x))

class ADown(nn.Module):
    def __init__(self, c1, c2): # ch_in, ch_out, shortcut, kernels, groups, expand
        super().__init__()
        self.c = c2 // 2
        self.cv1 = Conv(c1 // 2, self.c, 3, 2, 1)
        self.cv2 = Conv(c1 // 2, self.c, 1, 1, 0)
```



```
def forward(self, x):
    x = torch.nn.functional.avg_pool2d(x, 2, 1, 0, False, True)
    x1,x2 = x.chunk(2, 1)
    x1 = self.cv1(x1)
    x2 = torch.nn.functional.max_pool2d(x2, 3, 2, 1)
    x2 = self.cv2(x2)
    return torch.cat((x1, x2), 1)
```

5.3 Added improvement point 3 to the test.

Figure 12 shows the YOLOv8n+DWConv+CSPHet-CBAM+ADown on the Locount dataset.

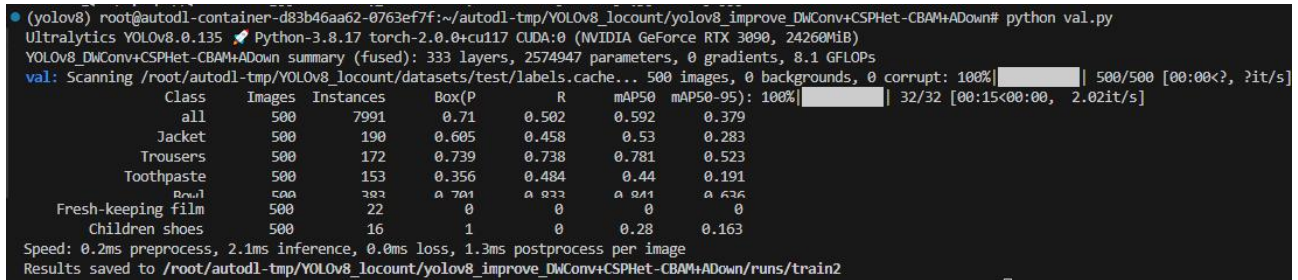


Figure 12 Results of YOLOv8n+DWConv+CSPHet-CBAM+ADown on the Locount dataset

6 Model compression (sparse training, channel pruning, model fine-tuning)

1. Channel pruning is a network compression technique that aims to reduce the computational and storage requirements of a model by removing redundant channels (i.e., the output channels of convolutional layers) in a neural network, while maintaining the model's performance as much as possible.

2. Assessing Channel Importance: The importance of each channel is measured using certain criteria (such as the channel's L1 norm, L2 norm, gradient magnitude, etc.). Generally, channels with lower importance contribute less to the network output and can therefore be considered for pruning.

3. Selective pruning:** Based on the importance assessment results, channels with minimal impact on model performance are selected for pruning. The purpose of pruning is to reduce the network's computational cost and memory usage, rather than significantly sacrificing accuracy.

4. Fine-tuning: After pruning, the model may lose some accuracy, so fine-tuning is needed to restore its performance. This process typically involves retraining the remaining parts of the network.

Overall, channels and their corresponding inputs and outputs can be pruned to generate a compressed model with lower complexity. Finally, the pruned model is fine-tuned to restore accuracy, compressing the model, optimizing its structure, improving real-time detection performance, and making it more suitable for deployment on edge computing devices without affecting the model's detection accuracy.

Adaptive pruning, a comparison table of different pruning rates (unadjusted), roughly captures the point of dramatic drop in mAP. The final improved-YOLOv8n, with a pruning rate of 0.5, channel number, and other metrics are shown in Table 2. The specific changes in the number of pruning channels are shown in Figure 13.

Table 2. Changes in channel count and other metrics for improved-YOLOv8n algorithm using a pruning rate of 0.5.

Pruning rate	Parameter (Million)	<u>GFLOPs</u>	Model size(MB)	<u>mAP50</u>	FPS
0	2.57	8.1	5.18	0.592	277.7
0.1	2.42	7.7	5.06	0.588	282.9
0.2	2.32	7.2	4.72	0.582	295.2
0.3	2.29	6.9	4.4	0.58	296.8
0.4	2.18	6.7	4.31	0.575	321.5
0.5	2.11	6.6	4.24	0.563	351.2
0.6	1.98	4.3	3.75	0.33	340.5
0.7	1.70	3.8	3.1	0.12	332.1
0.8	1.22	3.2	2.2	0.01	335.8
0.9	1.16	2.9	0.98	0.01	317.0

The pruning rate was set to 0.5. Before and after channel comparison images:

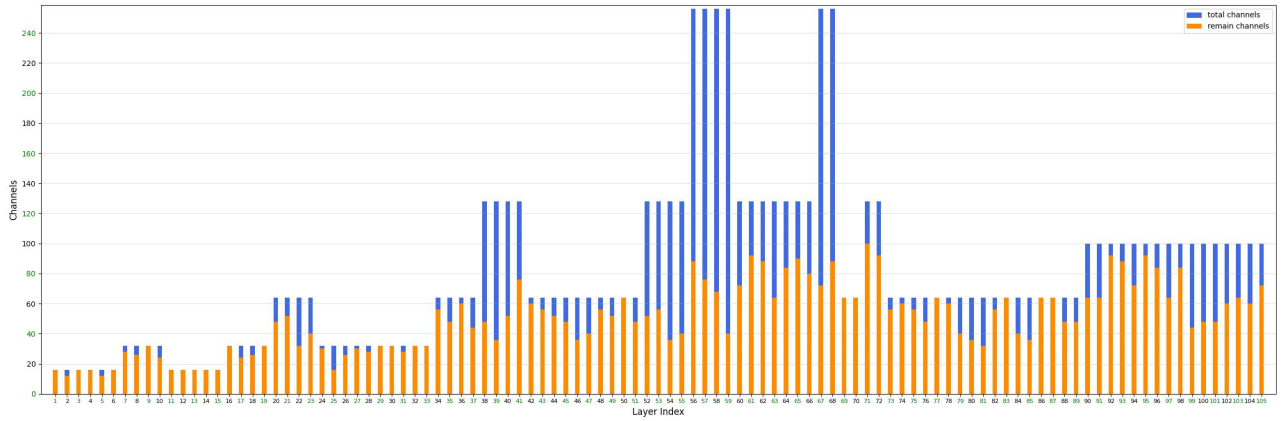


Figure 13 Changes in the number of channels in the YOLOv8n-Improved Model with a pruning rate of 0.5

The pruning rate was set to 0.5, and fine-tuning of the pruning was performed. After fine-tuning, the model's validation results on the test set are as follows: Comparison table 3 of the model before and after compression:

Table 3. Changes in various metrics of the Improved-YOLO v8n model before and after compression.

Model	Precision	Recall	mAP@50	mAP@50-95	parameters	GFLOPs	FPS
YOLOv8n+DWConv+CSPHet-CBAM+ADown	0.71	0.502	0.592	0.379	2574947	8.1	277.7
After model compression (<u>Lightweight-YOLO</u>)	0.741	0.649	0.722	0.478	2114768	6.6	357.1

7 Final model compression

Figure 14 shows the Lightweight-YOLO (Final YOLO) test results on the Locount dataset.

```

(yolov8) root@autodl-container-d83b46aa62-0763ef7f:~/autodl-tmp/YOLOv8_locount/yolov8_end_prune+quantize+distillation# python val.py
YOLOv8_DwConv+CSPHET-CBAM+ADown summary (fused): 333 layers, 2114768 parameters, 0 gradients, 6.6 GFLOPs
(333, 2114768, 0, 6.6062256)
Ultralytics YOLOv8.0.135 Python-3.8.17 torch-2.0.0+cu117 CUDA:0 (NVIDIA GeForce RTX 3090, 24260MiB)
val: Scanning /root/autodl-tmp/YOLOv8_locount/datasets/test/labels.cache... 500 images, 0 backgrounds, 0 corrupt: 100%|██████████| 500/500 [00:00<?, ?it/s]
Class Images Instances Box(P) R mAP50 mAP50-95: 100%|██████████| 32/32 [00:15<00:00, 2.12it/s]
all 500 7991 0.741 0.649 0.722 0.478
Jacket 500 190 0.744 0.506 0.654 0.394
Trousers 500 172 0.819 0.744 0.844 0.556
Toothpaste 500 153 0.622 0.542 0.617 0.36
Food box 500 6 1 0 0.00619 0.00371
Fresh-keeping film 500 22 0 0 0.0448 0.0265
Children shoes 500 16 0.0467 0.00584 0.542 0.255
Speed: 0.1ms preprocess, 2.0ms inference, 0.0ms loss, 0.7ms postprocess per image
Results saved to /root/autodl-tmp/YOLOv8_locount/yolov8_end_prune+quantize+distillation/runs/train2

```

Figure 14 Lightweight-YOLO (Final YOLO) test results on the Locount dataset

8 Heatmap Comparison

Figure 15 shows a comparative analysis of the heatmap results of Lightweight-YOLO and other algorithms.



Figure 15 Lightweight-YOLO (Final YOLO) and various algorithms in the Location dataset heatmap results

9 coco2017 generalization experiment

9.1 YOLOv8 algorithm baseline model

Figure 16 shows the YOLOv8n tested on the COCO2017 dataset.


```

(yolov8) root@autodl-container-d83b46aa62-0763ef7f:~/autodl-tmp/YOLOv8_coco2017/yolov8# python val.py
Ultralytics YOLOv8.0.229 Python-3.8.17 torch-2.0.0+cu117 CUDA:0 (NVIDIA GeForce RTX 3090, 24260MiB)
YOLOv8n summary (fused): 168 layers, 3151904 parameters, 0 gradients, 9.6 GFLOPs
val: Scanning ../datasets/coco/labels/val2017.cache... 5000 images, 48 backgrounds, 0 corrupt: 100%|██████████| 5000/5000 [00:00<?, ?it/s]
   Class  Images  Instances  Box(P)      R      mAP50  mAP50-95): 100%|██████████| 313/313 [01:59<00:00, 2.63it/s]
   all      5000      36781    0.604    0.446    0.493    0.342
 person    5000     11004    0.748    0.643    0.72     0.49
 bicycle   5000       316    0.618    0.37     0.417    0.237
   car      5000     1932    0.628    0.496    0.534    0.336
 motorcycle 5000       371    0.547    0.547    0.626    0.307
 toothbrush 5000       57     0.429    0.132    0.179    0.117
Speed: 1.3ms preprocess, 3.4ms inference, 0.0ms loss, 4.2ms postprocess per image
Saving runs/detect/val/predictions.json...

```

Figure 16 YOLOv8n tested on the COCO2017 dataset

9.2 DWConv

Figure 17 shows the YOLOv8n+DWConv tested on the COCO2017 dataset.

```

(yolov8) root@autodl-container-d83b46aa62-0763ef7f:~/autodl-tmp/YOLOv8_coco2017/yolov8_improve_DWConv# python val.py
Ultralytics YOLOv8.0.229 Python-3.8.17 torch-2.0.0+cu117 CUDA:0 (NVIDIA GeForce RTX 3090, 24260MiB)
YOLOv8n_DWConv summary (fused): 168 layers, 2969312 parameters, 0 gradients, 8.5 GFLOPs
val: Scanning ../datasets/coco/labels/val2017.cache... 5000 images, 48 backgrounds, 0 corrupt: 100%|██████████| 5000/5000 [00:00<?, ?it/s]
   Class  Images  Instances  Box(P)      R      mAP50  mAP50-95): 100%|██████████| 313/313 [01:55<00:00, 2.71it/s]
   all      5000      36781    0.586    0.457    0.491    0.338
 person    5000     11004    0.741    0.667    0.732    0.488
 bicycle   5000       316    0.625    0.377    0.433    0.234
   car      5000     1932    0.629    0.511    0.536    0.332
 motorcycle 5000       371    0.547    0.547    0.640    0.326
 toothbrush 5000       57     0.343    0.14     0.178    0.0977
Speed: 0.2ms preprocess, 1.9ms inference, 0.0ms loss, 2.9ms postprocess per image
Saving runs/detect/val/predictions.json...

```

Figure 17 YOLOv8n+DWConv tested on the COCO2017 dataset

9.3 DWConv+CSPHet-CBAM

Figure 18 shows the YOLOv8n+DWConv+CSPHet-CBAM tested on the COCO2017 dataset.

```

(yolov8) root@autodl-container-d83b46aa62-0763ef7f:~/autodl-tmp/YOLOv8_coco2017/yolov8_improve_DWConv+CSPHet-CBAM# python val.py
Ultralytics YOLOv8.0.229 Python-3.8.17 torch-2.0.0+cu117 CUDA:0 (NVIDIA GeForce RTX 3090, 24260MiB)
YOLOv8n_DWConv+CSPHet-CBAM summary (fused): 321 layers, 2666280 parameters, 0 gradients, 7.9 GFLOPs
val: Scanning ../datasets/coco/labels/val2017.cache... 5000 images, 48 backgrounds, 0 corrupt: 100%|██████████| 5000/5000 [00:00<?, ?it/s]
   Class  Images  Instances  Box(P)      R      mAP50  mAP50-95): 100%|██████████| 313/313 [00:48<00:00, 6.44it/s]
   all      5000      36781    0.592    0.45     0.487    0.326
 person    5000     11004    0.767    0.655    0.735    0.482
 bicycle   5000       316    0.673    0.359    0.424    0.23
   car      5000     1932    0.635    0.502    0.539    0.325
 motorcycle 5000       371    0.735    0.531    0.635    0.35
   airplane 5000      143    0.757    0.713    0.798    0.584
   bus       5000       205    0.700    0.660    0.744    0.567
 hair drier 5000       11       1       0       0.0638    0.0383
 toothbrush 5000       57     0.447    0.123    0.178    0.0931
Speed: 0.1ms preprocess, 2.3ms inference, 0.0ms loss, 0.8ms postprocess per image
Saving runs/detect/val/predictions.json...

```

Figure 18 YOLOv8n+DWConv+CSPHet-CBAM tested on the COCO2017 dataset

9.4 DWConv+CSPHet-CBAM+ADown

Figure 19 shows the YOLOv8n+DWConv+CSPHet-CBAM+ADown tested on the COCO2017 dataset.

```

(yolov8) root@autodl-container-d83b46aa62-0763ef7f:~/autodl-tmp/YOLOv8_coco2017/yolov8_improve_DWConv+CSPHet-CBAM+ADown# python val.py
Ultralytics YOLOv8.0.229 Python-3.8.17 torch-2.0.0+cu117 CUDA:0 (NVIDIA GeForce RTX 3090, 24260MiB)
YOLOv8n_DWConv+CSPHet-CBAM+ADown summary (fused): 333 layers, 2383400 parameters, 0 gradients, 7.2 GFLOPs
val: Scanning ../datasets/coco/labels/val2017.cache... 5000 images, 48 backgrounds, 0 corrupt: 100%|██████████| 5000/5000 [00:00<?, ?it/s]
   Class  Images  Instances  Box(P)      R      mAP50  mAP50-95): 100%|██████████| 313/313 [00:31<00:00, 9.86it/s]
   all      5000      36781    0.577    0.438    0.49     0.335
 person    5000     11004    0.764    0.644    0.735    0.492
 bicycle   5000       316    0.628    0.354    0.425    0.236
   car      5000     1932    0.627    0.464    0.524    0.326
 motorcycle 5000       371    0.719    0.504    0.621    0.361
   airplane 5000      143    0.758    0.734    0.788    0.6
   bus       5000       205    0.714    0.640    0.747    0.567
 hair drier 5000       11       0       0       0       0
 toothbrush 5000       57     0.443    0.105    0.2     0.105
Speed: 0.1ms preprocess, 2.4ms inference, 0.0ms loss, 0.6ms postprocess per image

```

Figure 19 YOLOv8n+DWConv+CSPHet-CBAM+ADown tested on the COCO2017 dataset

9.5 Model compression (sparse training, channel pruning, model fine-tuning)

The pruning retention rate was set to 0.5, and fine-tuning of the pruning was performed. The model test set validation results are Figure 20.

```
(yolov8) root@autodl-container-d83b46aa62-0763ef7f:~/autodl-tmp/YOLOv8_coco2017/yolov8_end_prune+quantize# python val.py
Ultralytics YOLOv8.0.229 Python-3.8.17 torch-2.0.0+cu117 CUDA:0 (NVIDIA GeForce RTX 3090, 24260MiB)
YOLOv8n summary (fused): 333 layers, 1768097 parameters, 0 gradients, 5.1 GFLOPs
val: Scanning ../datasets/coco/labels/val2017.cache... 5000 images, 48 backgrounds, 0 corrupt: 100% | 313/313 [00:31<00:00, 10.06it/s]
Class Images Instances Box(P) R mAP50 mAP50-95: 100% | 313/313 [00:31<00:00, 10.06it/s]
all 5000 36781 0.564 0.439 0.495 0.341
person 5000 11004 0.748 0.651 0.74 0.501
bicycle 5000 316 0.588 0.348 0.431 0.252
car 5000 1932 0.615 0.479 0.544 0.34
motorcycle 5000 371 0.745 0.526 0.647 0.386
airplane 5000 143 0.79 0.713 0.804 0.598
train 5000 706 0.777 0.66 0.756 0.617
teddy bear 5000 191 0.731 0.542 0.63 0.43
hair drier 5000 11 0 0 0 0
toothbrush 5000 57 0.542 0.104 0.203 0.0904
Speed: 0.1ms preprocess, 2.3ms inference, 0.0ms loss, 0.6ms postprocess per image
Saving runs/detect/val/predictions.json...
```

Figure 20 Lightweight-YOLO (Final YOLO) tested on the COCO2017 dataset

Comparison table before and after compression as show table 4:

Table 4 Performance comparison of the Lightweight-YOLO model before and after compression

Model	Precision	Recall	<u>mAP@50</u>	<u>mAP@50-95</u>	parameters	<u>GFLOPs</u>	FPS
YOLOv8n+DWConv+CSPHet-CBAM+ADown	0.577	0.438	0.49	0.335	2383400	7.2	322.5
After model compression (Lightweight-YOLO)	0.564	0.439	0.495	0.341	1768097	5.1	333.3

9.6 Model Comparison

YOLOv8n:

Execute the following command to verify:

```
cd /root/autodl-tmp/YOLOv8_coco2017/yolov8
python val.py
```

Figure 21 shows the YOLOv8n tested on the COCO2017 dataset (mAP).

```
(yolov8) root@autodl-container-d83b46aa62-0763ef7f:~/autodl-tmp/YOLOv8_coco2017/yolov8# python val.py
Ultralytics YOLOv8.0.229 Python-3.8.17 torch-2.0.0+cu117 CUDA:0 (NVIDIA GeForce RTX 3090, 24260MiB)
YOLOv8n summary (fused): 168 layers, 3151904 parameters, 0 gradients, 9.6 GFLOPs
val: Scanning ../datasets/coco/labels/val2017.cache... 5000 images, 48 backgrounds, 0 corrupt: 100% | 313/313 [01:59<00:00, 2.63it/s]
Class Images Instances Box(P) R mAP50 mAP50-95: 100% | 313/313 [01:59<00:00, 2.63it/s]
all 5000 36781 0.604 0.446 0.493 0.342
person 5000 11004 0.748 0.643 0.72 0.49
bicycle 5000 316 0.618 0.37 0.417 0.237
car 5000 1932 0.628 0.496 0.534 0.336
motorcycle 5000 371 0.547 0.476 0.507 0.307
airplane 5000 143 0.79 0.713 0.804 0.598
train 5000 706 0.777 0.66 0.756 0.617
teddy bear 5000 191 0.731 0.542 0.63 0.43
hair drier 5000 11 0 0 0 0
toothbrush 5000 57 0.429 0.132 0.179 0.117
Speed: 1.3ms preprocess, 3.4ms inference, 0.0ms loss, 4.2ms postprocess per image
Saving runs/detect/val/predictions.json...
```

Figure 21 YOLOv8n tested on the COCO2017 dataset (mAP)

SSD300

Commands for validation set verification, model parameter computation, and FPS acquisition:


```
cd /root/autodl-tmp/mmdetection_coco2017

# verify
python tools/test.py ssd300-config/ssd300_coco.py ssd300-train/epoch_12.pth --out=ssd300_resulttfcos.pkl

# Get Precision
python tools/analysis_tools/confusion_matrix.py ssd300-config/ssd300_coco.py ssd300_resulttfcos.pkl ./

# Model parameter computation cost acquisition
python tools/analysis_tools/get_flops.py ssd300-config/ssd300_coco.py

# FPS calculation
python tools/analysis_tools/benchmark.py ssd300-config/ssd300_coco.py --checkpoint ssd300-train/epoch_12.pth
```

```
# verify
```

```
python tools/test.py ssd300-config/ssd300_coco.py ssd300-train/epoch_12.pth --out=ssd300_resulttfcos.pkl
```

Get Precision

```
python tools/analysis_tools/confusion_matrix.py ssd300-config/ssd300_coco.py ssd300_resulttfcos.pkl ./
```

Model parameter computation cost acquisition

```
python tools/analysis_tools/get_flops.py ssd300-config/ssd300_coco.py
```

```
# FPS calculation
```

```
python tools/analysis_tools/benchmark.py ssd300-config/ssd300_coco.py --checkpoint ssd300-train/epoch_12.pth
```

Figure 22 shows the SSD tested on the COCO2017 dataset (mAP).

Average Precision	(AP)	@ IoU=0.50:0.95	area= all	maxDets=100	= 0.217
Average Precision	(AP)	@ IoU=0.50	area= all	maxDets=1000	= 0.398
Average Precision	(AP)	@ IoU=0.75	area= all	maxDets=1000	= 0.218
Average Precision	(AP)	@ IoU=0.50:0.95	area= small	maxDets=1000	= 0.058
Average Precision	(AP)	@ IoU=0.50:0.95	area= medium	maxDets=1000	= 0.236
Average Precision	(AP)	@ IoU=0.50:0.95	area= large	maxDets=1000	= 0.353
Average Recall	(AR)	@ IoU=0.50:0.95	area= all	maxDets=100	= 0.341
Average Recall	(AR)	@ IoU=0.50:0.95	area= all	maxDets=300	= 0.342
Average Recall	(AR)	@ IoU=0.50:0.95	area= all	maxDets=1000	= 0.342
Average Recall	(AR)	@ IoU=0.50:0.95	area= small	maxDets=1000	= 0.112
Average Recall	(AR)	@ IoU=0.50:0.95	area= medium	maxDets=1000	= 0.383
Average Recall	(AR)	@ IoU=0.50:0.95	area= large	maxDets=1000	= 0.519

```
creating index...  
index created!  
[>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>] 5000/5000, 575.9 task/s, elapsed: 9s,  
AP 0.6328989565892335  
F1 0.4155957760620543
```

```
data_preprocessor
=====
Use size divisor set input shape from (640, 481) to (300, 300)
=====
Compute type: dataloader: load a picture from the dataset
Input shape: (300, 300)
Flops: 34.36G
Params: 34.305M
=====
```

```
01/24 18:23:58 - mmengine - INFO - =====
01/24 18:23:58 - mmengine - INFO - Done image [950/1000], fps: 110.0 img/s, times per image: 9.1 ms/img, cuda memory: 271 MB
01/24 18:23:58 - mmengine - INFO - (GB) mem_used: 110.05 | uss: 2.88 | pss: 2.88 | total_proc: 1
01/24 18:23:59 - mmengine - INFO - =====
01/24 18:23:59 - mmengine - INFO - Done image [1000/1000], fps: 110.1 img/s, times per image: 9.1 ms/img, cuda memory: 271 MB
01/24 18:23:59 - mmengine - INFO - (GB) mem_used: 110.11 | uss: 2.88 | pss: 2.88 | total_proc: 1
01/24 18:23:59 - mmengine - INFO - ===== Done =====
01/24 18:23:59 - mmengine - INFO - Overall fps: 110.1 img/s, times per image: 9.1 ms/img
01/24 18:23:59 - mmengine - INFO - cuda memory: 134 MB
01/24 18:23:59 - mmengine - INFO - (GB) mem used: 110.12 | uss: 2.88 | pss: 2.88 | total proc: 1
```

Figure 22 SSD tested on the COCO2017 dataset (mAP)

Faster-RCNN

Commands for validation set verification, model parameter computation, and FPS acquisition:

YOLOv5

Execute the following command to validate the validation set:

```
cd /root/autodl-tmp/coco2017_contrast/yolov5

python val.py
```

Figure 24 shows the YOLOv5 tested on the COCO2017 dataset (mAP).

```
Fusing layers...
YOLOv5s summary: 157 layers, 7225885 parameters, 0 gradients, 16.4 GFLOPs
val: Scanning /root/autodl-tmp/YOLOv8_coco2017/datasets/coco/val2017.cache... 5000 images, 48 backgrounds, 0 corrupt: 100%|██████████| 5000/5000 [00:00<?, ?it/s]
      Class      Images  Instances      P      R      mAP50      mAP50-95: 100%|██████████| 157/157 [00:31<00:00, 5.02it/s]
      all         5000       36781      0.583    0.442    0.487      0.329
Speed: 0.1ms pre-process, 1.5ms inference, 1.9ms NMS per image at shape (32, 3, 640, 640)

Evaluating pycocotools mAP... saving runs/val/exp/best_predictions.json...
loading annotations into memory...
Done (t=0.35s)
creating index...
index created!
Loading and preparing results...
DONE (t=3.28s)
creating index...
index created!
Running per image evaluation...
Evaluate annotation type *bbox*
DONE (t=45.64s)
Accumulating evaluation results...
DONE (t=7.30s)
Average Precision (AP) @[ IoU=0.50:0.95 | area= all | maxDets=100 ] = 0.329
Average Precision (AP) @[ IoU=0.50 | area= all | maxDets=100 ] = 0.487
Average Precision (AP) @[ IoU=0.75 | area= all | maxDets=100 ] = 0.367
Average Precision (AP) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = 0.175
Average Precision (AP) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] = 0.379
Average Precision (AP) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.433
Average Recall (AR) @[ IoU=0.50:0.95 | area= all | maxDets= 1 ] = 0.287
Average Recall (AR) @[ IoU=0.50:0.95 | area= all | maxDets= 10 ] = 0.475
Average Recall (AR) @[ IoU=0.50:0.95 | area= all | maxDets=100 ] = 0.540
Average Recall (AR) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = 0.332
Average Recall (AR) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] = 0.599
Average Recall (AR) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.673
Results saved to runs/val/exp
```

Figure 24 YOLOv5 tested on the COCO2017 dataset (mAP)

YOLOv7

Execute the following command to validate the validation set:

```
cd /root/autodl-tmp/coco2017_contrast/yolov7

python test.py
```

Figure 25 shows the YOLOv7 tested on the COCO2017 dataset (mAP).

```
Model Summary: 208 layers, 6221370 parameters, 0 gradients, 13.7 GFLOPs
Convert model to Traced-model...
traced_script_module saved!
model is traced!
val: Scanning '/root/autodl-tmp/YOLOv8_coco2017/datasets/coco/val2017.cache' images and labels... 5000 found, 0 missing, 48 empty, 0 corrupted: 100%|██████████| 5000/5000 [00:00<?, ?it/s]
      Class      Images      Labels      P      R      mAP@.5      mAP@.5: .95: 100%|██████████| 157/157 [00:39<00:00, 4.00it/s]
      all         5000       36781      0.57    0.482    0.483      0.324
Speed: 1.8/0.9/2.7 ms inference/NMS/total per 640x640 Image at batch-size 32

Evaluating pycocotools mAP... saving runs/test/exp/best_predictions.json...
loading annotations into memory...
Done (t=0.80s)
creating index...
index created!
Loading and preparing results...
DONE (t=0.70s)
creating index...
index created!
Running per image evaluation...
Evaluate annotation type *bbox*
DONE (t=24.00s)
Accumulating evaluation results...
DONE (t=4.73s)
Average Precision (AP) @[ IoU=0.50:0.95 | area= all | maxDets=100 ] = 0.324
Average Precision (AP) @[ IoU=0.50 | area= all | maxDets=100 ] = 0.483
Average Precision (AP) @[ IoU=0.75 | area= all | maxDets=100 ] = 0.351
Average Precision (AP) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = 0.167
Average Precision (AP) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] = 0.360
Average Precision (AP) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.438
Average Recall (AR) @[ IoU=0.50:0.95 | area= all | maxDets= 1 ] = 0.269
Average Recall (AR) @[ IoU=0.50:0.95 | area= all | maxDets= 10 ] = 0.416
Average Recall (AR) @[ IoU=0.50:0.95 | area= all | maxDets=100 ] = 0.434
Average Recall (AR) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = 0.226
Average Recall (AR) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] = 0.478
Average Recall (AR) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.585
Results saved to runs/test/exp
```

Figure 25 YOLOv7 tested on the COCO2017 dataset (mAP)

YOLO11

Execute the following command to validate the validation set:

```
cd /root/autodl-tmp/coco2017_contrast/yolo11

python val.py
```

Figure 26 shows the YOLOv11 tested on the COCO2017 dataset (mAP).

```
(yolov8) root@autodl-container-d5044a90db-14048abb:~/autodl-tmp/coco2017_contrast/yolo11# python val.py
Ultralytics 8.3.23 Python-3.8.17 torch-2.0.0+cu117 CUDA:0 (NVIDIA GeForce RTX 3090, 24253MiB)
YOLO11n summary (fused): 238 layers, 2,616,248 parameters, 0 gradients, 6.5 GFLOPs
val: Scanning /root/autodl-tmp/YOLOv8_coco2017/datasets/coco/labels/val2017.cache... 5000 images, 48 backgrounds, 0 corrupt: 100%|██████████| 5000/5000 [00:00<?, ?it/s]
   Class      Images  Instances  Box(P          R          mAP50  mAP50-95): 100%|██████████| 313/313 [00:27<00:00, 11.27it/s]
   all         5000      36781      0.601      0.442      0.484      0.346
   person      2693      11004      0.718      0.625      0.699      0.484
   bicycle      149        316      0.602      0.38      0.393      0.222
   car          535      1932      0.609      0.474      0.515      0.333
   motorcycle   150        371      0.60      0.438      0.466      0.307
   toothbrush   34         57      0.421      0.158      0.155      0.094
Speed: 0.1ms preprocess, 2.5ms inference, 0.0ms loss, 0.6ms postprocess per image
Saving runs/detect/val/predictions.json...

Evaluating pycocotools mAP using runs/detect/val/predictions.json and /root/autodl-tmp/YOLOv8_coco2017/datasets/coco/annotations/instances_val2017.json...
loading annotations into memory...
Done (t=0.35s)
creating index...
index created!
Loading and preparing results...
DONE (t=6.68s)
creating index...
index created!
Running per image evaluation...
Evaluate annotation type *bbox*
DONE (t=57.40s).
Accumulating evaluation results...
DONE (t=14.24s).
Average Precision  (AP) @[ IoU=0.50:0.95 | area=   all | maxDets=100 ] = 0.346
Average Precision  (AP) @[ IoU=0.50      | area=   all | maxDets=100 ] = 0.484
Average Precision  (AP) @[ IoU=0.75      | area=   all | maxDets=100 ] = 0.380
Average Precision  (AP) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = 0.162
Average Precision  (AP) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] = 0.381
Average Precision  (AP) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.495
Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets=  1 ] = 0.304
Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets= 10 ] = 0.514
Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets=100 ] = 0.581
Average Recall     (AR) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = 0.337
Average Recall     (AR) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] = 0.646
Average Recall     (AR) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.773
Results saved to runs/detect/val
```

Figure 26 YOLOv11 tested on the COCO2017 dataset (mAP)

Final model compression: Figure 27 shows the Lightweight-YOLO (Final YOLO) tested on the COCO2017 dataset (mAP).

```
(yolov8) root@autodl-container-d83b46aa62-0763ef7f:~/autodl-tmp/YOLOv8_coco2017/yolov8_end_prune+quantize# python val.py
Ultralytics YOLOv8.0.229 Python-3.8.17 torch-2.0.0+cu117 CUDA:0 (NVIDIA GeForce RTX 3090, 24260MiB)
YOLOv8_DWConv+CSPHNet-CBAH+ADown summary (fused): 333 layers, 1768097 parameters, 0 gradients, 5.1 GFLOPs
val: Scanning ../datasets/coco/labels/val2017.cache... 5000 images, 48 backgrounds, 0 corrupt: 100%|██████████| 5000/5000 [00:00<?, ?it/s]
   Class      Images  Instances  Box(P          R          mAP50  mAP50-95): 100%|██████████| 313/313 [00:31<00:00, 10.06it/s]
   all         5000      36781      0.564      0.439      0.495      0.341
   person      5000      11004      0.748      0.651      0.74      0.501
   bicycle      5000        316      0.588      0.348      0.431      0.252
   car          5000      1932      0.615      0.479      0.544      0.34
   motorcycle   5000        371      0.745      0.526      0.647      0.386
   airplane     5000        143      0.79      0.713      0.804      0.598
   bear         5000        191      0.731      0.542      0.63      0.43
   teddy bear    5000        11      0      0      0      0
   hair drier    5000         57      0.542      0.104      0.203      0.0904
   toothbrush    5000         57      0.542      0.104      0.203      0.0904
Speed: 0.1ms preprocess, 2.3ms inference, 0.0ms loss, 0.6ms postprocess per image
Saving runs/detect/val/predictions.json...
```

Figure 27 Lightweight-YOLO (Final YOLO) tested on the COCO2017 dataset (mAP)

Table5 Comparison results of Lightweight-YOLO with other algorithms on the COCO2017 dataset

	Precision	Recall	<u>mAP50</u>	<u>mAP50-95</u>	parameters	<u>GFLOPs</u>	FPS
<u>YOLOv8n</u>	0.604	0.446	0.493	0.342	3151904	9.6	112.3
<u>SSD300</u>	0.632	0.341	0.398	0.217	34305000	34.3	110.1
Faster-RCNN	0.660	0.383	0.486	0.325	41753000	187.0	31.8
YOLOv5s	0.583	0.442	0.487	0.329	7225885	16.4	289.4
YOLOv7	0.57	0.482	0.483	0.324	6221370	13.7	185.1
YOLO11n	0.601	0.442	0.484	0.346	2616248	6.5	318.7
Final model pruning(Lightweight-YOLO)	0.564	0.439	0.495	0.341	1768097	5.1	333.3

Comparison results of Lightweight-YOLO with other algorithms on the COCO2017 dataset are shown in Table 5. Figure 27 shows the heatmap results of Lightweight-YOLO compared with various algorithms on the COCO2017 dataset.

9.7 Heatmap Comparison

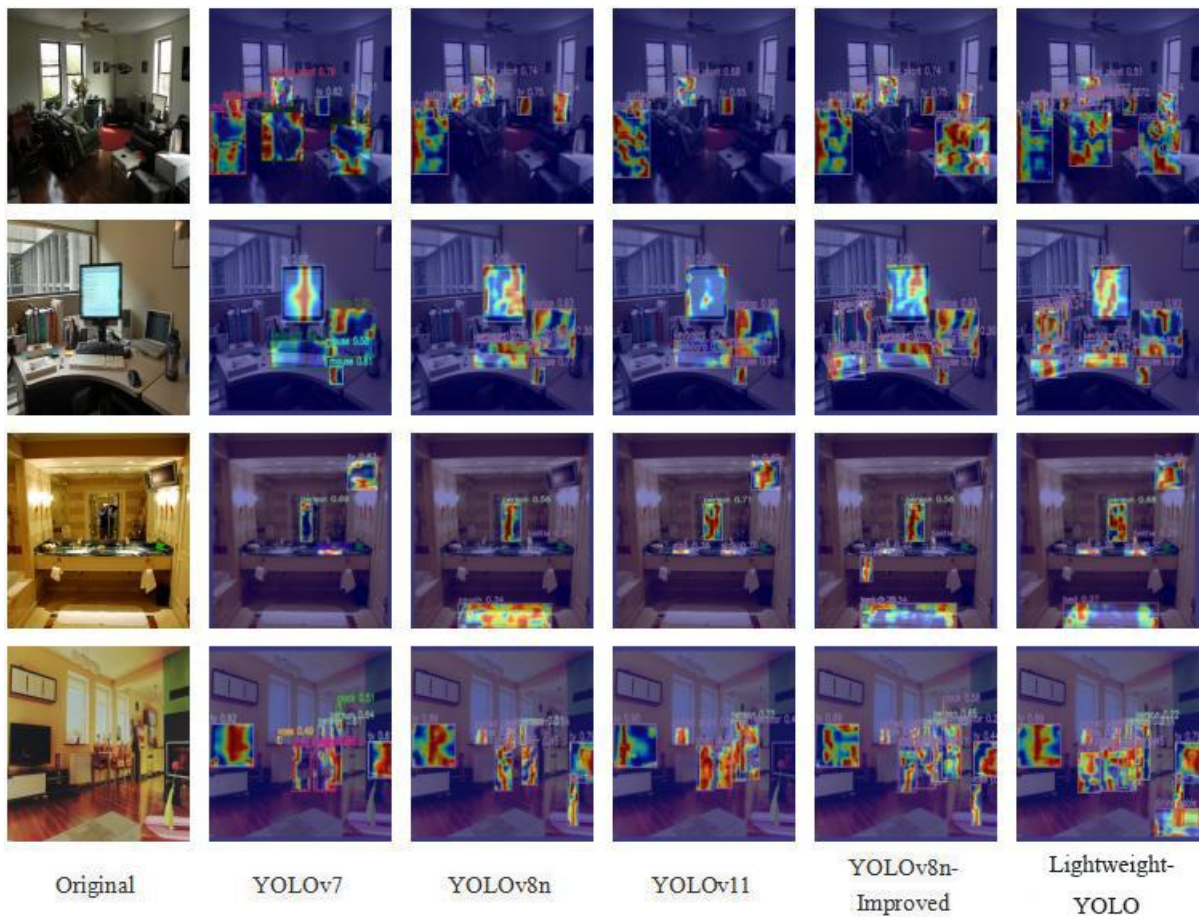


Figure 28 Lightweight-YOLO (Final YOLO) and various algorithms in the COCO2017 dataset heatmap results